

Resumen del proyecto Presupuestados Back

Fecha de elaboracion: 2026-03-14

Vision general

Presupuestados Back es una API REST construida con NestJS y TypeScript para soportar una aplicacion de gestion financiera enfocada en parejas u hogares. El backend centraliza autenticacion, perfiles, vinculacion entre miembros, registro de gastos, ingresos, presupuestos, deducciones, dashboard consolidado y funciones asistidas por IA.

La aplicacion usa Supabase como plataforma principal de datos y autenticacion. El backend opera con una service role key para ejecutar consultas directas sobre la base de datos, mientras que la identidad del usuario se valida con un guard que verifica el Bearer token contra Supabase Auth.

Stack tecnico

- Framework principal: NestJS 11
- Lenguaje: TypeScript
- Runtime: Node.js
- Base de datos y auth: Supabase
- Documentacion de API: Swagger en '/api'
- Validacion de entrada: 'class-validator' y 'ValidationPipe'
- IA generativa: Google Gemini mediante '@google/genai'
- Carga de archivos: 'multer'

Arquitectura general

El proyecto sigue una arquitectura modular de NestJS. AppModule integra los modulos funcionales y main.ts configura validacion global y documentacion Swagger.

Modulos principales

- 'AuthModule': registro, login, refresh, logout e inicializacion del usuario.
- 'CouplesModule': union de usuarios a una pareja mediante codigo de invitacion.
- 'ProfilesModule': consulta y actualizacion del perfil autenticado.
- 'ExpensesModule': CRUD de gastos, soporte para gastos recurrentes y eliminacion por lote.
- 'IncomesModule': CRUD de ingresos del hogar.
- 'BudgetsModule': CRUD de presupuestos.
- 'DeductionsModule': CRUD de deducciones.
- 'DashboardModule': entrega consolidada de miembros, ingresos, deducciones, presupuestos y gastos.
- 'AIModule': procesamiento de cartolas o estados de cuenta para extraer gastos automaticamente.
- 'ChatbotModule': asistente conversacional con acceso controlado a datos financieros.
- 'SupabaseModule': encapsula la creacion del cliente tipado de Supabase.

Flujo de autenticacion y seguridad

- El backend espera tokens Bearer emitidos por Supabase Auth.
- 'AuthGuard' extrae el token del header 'Authorization', valida el usuario con 'supabase.auth.getUser(token)' y adjunta 'req.user'.
- La aplicacion usa 'ValidationPipe' global con 'whitelist', 'forbidNonWhitelisted' y 'transform'.
- La mayoría de los endpoints de negocio requieren autenticacion.
- El cliente de Supabase se inicializa con 'SUPABASE_SERVICE_KEY', por lo que el control de acceso depende fuertemente de la validacion realizada en NestJS.

Integracion con Supabase

SupabaseService crea un cliente tipado usando database.types.ts. La intencion del proyecto es concentrar toda la logica de negocio en el backend y evitar que el frontend consuma la base de datos directamente.

Variables de entorno detectadas:

- 'SUPABASE_URL'
- 'SUPABASE_SERVICE_KEY'
- 'PORT'

Ademas, los modulos de IA requieren GEMINI_API_KEY, aunque esa variable no aparece en .env.example.

Funcionalidades implementadas

1. Autenticacion y onboarding

Endpoints detectados:

- 'POST /auth/register'
- 'POST /auth/login'
- 'POST /auth/refresh'
- 'POST /auth/logout'
- 'POST /auth/initialize'

Este modulo se encarga de crear usuarios, iniciar sesion, renovar tokens y completar la inicializacion de datos del usuario recién creado.

2. Gestion de parejas y perfiles

Endpoints detectados:

- 'POST /couples/join'
- 'GET /profiles/me'
- 'PUT /profiles/me'

La logica apunta a asociar usuarios a una unidad familiar o pareja y permitir la administracion de su perfil principal.

3. Gestion financiera central

Gastos

Endpoints detectados:

- 'GET /expenses'
- 'POST /expenses'
- 'PUT /expenses/:id'
- 'PUT /expenses/recurring/:id'
- 'PUT /expenses/recurring/:id/stop'
- 'DELETE /expenses/batch'
- 'DELETE /expenses/:id'

El modulo de gastos es uno de los nucleos del sistema. Soporta operaciones CRUD, manejo de recurrencia y eliminacion por lotes, lo que calza con la importacion automatizada desde cartolas.

Ingresos

Endpoints detectados:

- 'GET /incomes'
- 'POST /incomes'
- 'PUT /incomes/:id'
- 'DELETE /incomes/:id'

Presupuestos

Endpoints detectados:

- 'GET /budgets'
- 'POST /budgets'
- 'PUT /budgets/:id'
- 'DELETE /budgets/:id'

Deducciones

Endpoints detectados:

- 'GET /deductions'
- 'POST /deductions'
- 'PUT /deductions/:id'
- 'DELETE /deductions/:id'

4. Dashboard consolidado

Endpoint detectado:

- 'GET /dashboard'

DashboardService consulta en paralelo las tablas family_members, incomes, deductions, budgets y expenses usando Promise.all, devolviendo un payload unico para la carga inicial del frontend.

5. Funcionalidades con IA

Procesamiento de cartolas

Endpoint detectado:

- 'POST /ai/process-statement'

Flujo identificado:

- Verifica que el usuario sea premium.
- Obtiene 'couple_id' y el 'family_member' asociado.
- Consulta categorias desde la base.
- Envía una imagen o PDF a Gemini para extraer gastos en JSON.
- Inserta el resultado como lote en la tabla 'expenses'.

Esta funcionalidad permite acelerar el ingreso de gastos desde documentos bancarios.

Chatbot financiero

Endpoint detectado:

- 'POST /chatbot/chat'

El chatbot utiliza Gemini con declaraciones de funciones para consultar o modificar datos bajo demanda. Las herramientas definidas actualmente permiten:

- obtener gastos mensuales,
- obtener ingresos mensuales,
- actualizar la categoria de un gasto.

El enfoque reduce la necesidad de cargar toda la informacion financiera en el prompt inicial y delega consultas puntuales a servicios internos.

Documentacion y experiencia de desarrollo

- Swagger se expone en '/api'.
- Scripts npm disponibles: 'build', 'start', 'start:dev', 'start:prod', 'lint', 'test', 'test:cov', 'test:e2e'.
- Existen pruebas unitarias visibles principalmente en el modulo de 'expenses' y en el controlador principal.
- El 'README.md' actual sigue siendo el template por defecto de NestJS, por lo que no documenta todavia el dominio de negocio real del proyecto.

Hallazgos relevantes

- El proyecto ya refleja gran parte del plan definido en 'implementation_plan.md'.
- La API esta pensada como fachada completa sobre Supabase para que el frontend no interactue directamente con la base.
- El dominio del sistema esta claramente orientado a finanzas compartidas: pareja, miembros del hogar, prorrato y control de presupuesto.
- Hay una combinacion de capacidades transaccionales classicas y funciones premium apoyadas por IA.
- '.env.example' no documenta 'GEMINI_API_KEY', aunque el codigo si la exige para 'AIModule' y 'ChatbotModule'.

Posibles mejoras

- Reemplazar el 'README.md' generico por documentacion especifica del proyecto.
- Centralizar la logica repetida para obtener 'couple_id', ya que aparece en varios controladores.
- Documentar mejor el modelo de datos esperado en Supabase y las tablas involucradas.
- Ampliar la cobertura de pruebas en modulos fuera de 'expenses'.
- Agregar ejemplos de peticiones y respuestas para facilitar integracion con frontend o terceros.

Conclusion

Presupuestados Back es un backend modular y bien encaminado para una aplicacion de finanzas compartidas. Combina operaciones CRUD tradicionales, agregacion de informacion para dashboard y automatizaciones con IA sobre una base de datos en Supabase. El proyecto ya tiene una base funcional solida, aunque todavia le vendria bien fortalecer su documentacion tecnica y estandarizar algunos patrones internos.